林华钟

13715162826 942701380@qq.com

男 应届生 CET-6

教育背景

华南师范大学（211） 计算机技术 (硕士) 2024.09 - 2027.07

- [1] **LIN H Z**, ZHENG Q A, et al. Set transformer guided NSGA-III for large-scale multi-objective optimization.(ISICA 2025)

惠州学院（本科） 电子信息工程 (本科) 2017.09 - 2021.07

专业技能&个人优势

- **Agent 核心研发与评测：**深入掌握 ReAct、MCP、Skills、多 Agent 协作与上下文工程（工具动态延迟加载/渐进式压缩）；熟练基于 Harbor 搭建自动化评测与 Pytest 回归测试流水线，具备独立自研终端 Coding Agent 经验。
- **本地化 RAG 与检索工程：**能独立构建受限显存下的全本地化 RAG 问答系统，熟练掌握扫描件图像增强与版面分析（PaddleOCR）、向量+关键词混合检索与重排序（Milvus + ES + BGE + RRF）、Ollama 多模型动态调度及低幻觉调优。
- **全栈产品独立交付：**熟练使用 Python (FastAPI)、TypeScript/JavaScript、MATLAB；兼具 React、Vue 3、Electron 及微信原生小程序工程实战能力，可独立完成从底层智能体/微服务到桌面端/Web 端流式交互界面的端到端交付。

开源贡献

Magpie (GitHub**30k+★**) | **Contributor** — AI Coding Agent 统一模型网关与调度工具（**https://github.com/yetone/magpie**）

- **测试沙箱隔离治理：**修复专用目录环境变量绕过测试临时 Home 的隔离缺陷，通过临时会话与配置目录验证数据误写行为；
- **跨模块环境配置解耦：**抽取共享 agentenv 清单并接入 21 处通用沙箱，处理跨包依赖及 fixture 设置顺序；
- **网关路由与配置静态校验：**补齐 usage/provider 的 TestMain 入口，加入变量清单引用、重复和源码读取归属检查；

项目经历

南方电网某供电局--智能检索知识问答系统 **全栈开发** 2025.02 - 2025.07

项目介绍： 面向供电局政策、标准文件（大量为扫描件）做全本地化的检索问答。在已有 Electron + Vue 3 + Go gRPC 文件管理客户端上，独立新建 Python RAG 服务层。

技术栈：Python、FastAPI、Milvus、Elasticsearch、PaddleOCR、BGE、Ollama、Electron、Vue 3

- **8GB 显存下的本地模型调度：** 对话模型、Embedding、OCR、Reranker 无法同时常驻显存。我设计了按场景轮换的方案：问答时用 Ollama 加载 Qwen2.5-7B，入库时切换到 Embedding；PaddleOCR 与 BGE-Reranker 固定跑 CPU，不占显存。**单次模型切换 3-5 秒，单用户问答端到端约 10 秒。**
- **扫描件批量入库：** 政策文件多数为扫描件，直接 OCR 错字多。设计四步图像预处理（**Hough 倾斜校正 → NLMeans 去噪 → 自适应二值化 → CLAHE 增强**）接入PaddleOCR PP-StructureV2完成版面分析、文字识别与表格结构化，每段文字保留 OCR 置信度，检索时低置信度片段降权，避免错字内容排到前面。CPU 8线程并行，**100页扫描文档30-50秒**；
- **混合检索与效果优化：** BGE-large-zh 向量 + ES BM25，用 RRF 合并名次，再用 BGE-Reranker 精排。同时利用原系统的专业/政策分类，在 Milvus 里先做标量过滤，将候选范围缩小 60%-80%。在自建约 200 题评估集的 150 道可作答题上把 **Recall@5 提升至 95%+、MRR 提升至 85%+ ，答案覆盖度 97%，人工逐条核过的幻觉率 0.7%；**
- **流式问答与权限控制：** FastAPI 用 SSE 推流，Electron 主进程转发给渲染进程，前端边收边渲染 Markdown，答案旁可展开原文片段。检索时按用户角色的 auth_level 过滤，**不同角色仅可检索授权范围内的文档片段；**

Hcode--Coding Agent **Agent研发** 2026.04 - 至今

项目介绍：面向终端场景的轻量级 Coding Agent，基于 ReAct 循环驱动 LLM 自主完成代码检索、编辑与测试闭环。采用五层分层架构，支持多模型接入、MCP 工具扩展与多 Agent 协作。独立完成各模块设计与开发，核心模块先写 Pytest 用例再写实现。

技术栈：Python、ReAct、MCP、Skills、Multi-Agent、Pytest、Harbor

- **上下文与Token管理**： 启动时只加载工具名与简介，调用时再取完整定义，**工具描述 Token 占用降低 80% 以上**；设计**两层渐进式上下文压缩策略**，第一层负责裁剪工具调用返回较大结果，第二层对触发阈值上下文进行压缩，长时间连续编程会话仍保留任务目标、已改动文件等关键上下文；
- **权限与安全隔离**： 按操作风险分层，低风险操作自动放行，**单次会话权限弹窗由平均 20 次降到 5 次以内**，避免用户审批疲劳后一路点"同意"。**可选启用内核级隔离**，Agent 全自动模式下也能把风险限制在沙箱内；
- **评测驱动优化：**基于 Harbor 搭建 installed-agent 评测流程，**terminal-bench-sample@2.0 任务完成率 70%** 。自建 hidden-pytest 回归集，逐个分析失败任务的日志，定位并修复长耗时依赖安装超时、服务启动后未校验就结束任务等问题；
- **多 Agent 并行协作**： 跨文件重构时按文件拆分给多个子 Agent 并行处理，每个只改自己负责的文件，避免写冲突，同时突破单 Agent 的上下文窗口限制。**跨文件重构耗时降低约 50%**；